

THE UNIVERSITY OF
MELBOURNE

Final Report

The Art of Scientific Computing: Project Title: High Dimensional Kalman Filters

Author:
Steven Pavli

Subject Handbook code:
COMP-90072

Faculty of Science
The University of Melbourne

Original Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Todo . . . 1
2 Introduction . . . 1
3 Background . . . 1
4 Dynamic Projections . . . 2
5 Dynamic PCA-KF . . . 3
6 Results . . . 3
7 Discussion . . . 6
1 AI Declaration . . . 7
2 Reproducing results . . . 7

Bibliography **7**

1 Todo

- Evaluation: Describe RMSE for y, beta.
- Finish discussion
- Code stuff: README.md, quick error check (small-sample run_simulation.py and rotation_test.py)

2 Introduction

A Kalman filter (KF) is a type of algorithm which aims to estimate a hidden state in a linear system, given a series of observations. In particular, it handles the case in which the state variable evolves over time, and does so in a mean-squared optimal manner. This project aims to implement such an algorithm in the case of high dimensional linear regression, via the use of principle components analysis (PCA). The combination of PCA and KF exists in the literature, however, the papers largely deal with a one-time PCA fits, rather than updating representations over time. To extend this, we experiment with a new type of rotating covarinace structure, and test a way of keeping the filter in the correct space via a method we term *projection chaining*. We compare projection chaining to the usual static approach and discuss some properties and limitations of the method.

3 Background

We begin by providing background on the standard KF; its optimality and time complexity. We will present the theory in the case of linear regression by taking a special case of the original construction of [2]. Equation 1 is the state process equation which governs the dynamics of the unobserved state $\boldsymbol{\beta}_t \in \mathbb{R}^p$. Equation 2 is the observation process, which describes the relationship between observation $y_t \in \mathbb{R}$ and other observed data $\boldsymbol{x}_t \in \mathbb{R}^p$.

$$\boldsymbol{\beta}_t = \boldsymbol{\beta}_{t-1} + \boldsymbol{\varepsilon}_t \tag{1}$$

$$y_t = \boldsymbol{\beta}_t^T \boldsymbol{x}_t + u_t \tag{2}$$

where $\boldsymbol{\varepsilon}_t \sim N(0, \boldsymbol{Q})$, and $u_t \sim N(0, R)$. Here, $\boldsymbol{Q} \in \mathbb{R}^{p \times p}$ is the state noise covariance matrix, and R is the measurement noise (co)variance, both of which are assumed to be fixed over time in this project.

We denote the current estimate of $\boldsymbol{\beta}_t$ as of time $s \leq t$ as $\hat{\boldsymbol{\beta}}_{t|s}$. Further, we assume we have an initial state estimate $\boldsymbol{\beta}_{1|1}$; we describe a method of estimating this in Section 5. Define $\boldsymbol{P}_{t|s} = \mathbb{E}[(\boldsymbol{\beta}_t - \boldsymbol{\beta}_{t|s})(\boldsymbol{\beta}_t - \boldsymbol{\beta}_{t|s})^T] \in \mathbb{R}^{p \times p}$ to be the estimate error covariance.

At time t, we predict $\boldsymbol{\beta}_{t|t-1} = \boldsymbol{\beta}_{t-1|t-1}$, and $\boldsymbol{P}_{t|t-1} = \boldsymbol{P}_{t-1|t-1} + \boldsymbol{Q}$, then estimate what $\boldsymbol{y}_t$ would be under this state, given $\boldsymbol{x}_t$;

$$\hat{y}_t = \boldsymbol{\beta}_{t|t-1}\boldsymbol{x}_t$$

then calculate the residual: $\tilde{y}_t = y_t - \hat{y}_t$. The central idea of the KF is that we assume $\boldsymbol{\beta}_{t|t} = \boldsymbol{\beta}_{t|t-1} + \boldsymbol{K}_t\tilde{y}_t$ where $\boldsymbol{K}_t$ is the *Kalman gain* at time t. Specifically, $\boldsymbol{K}_t$ is chosen to minimise $\mathbb{E}(\|\boldsymbol{\beta}_t - \boldsymbol{\beta}_{t|t}\|^2)$, and has closed form. Furthermore, we also update $\boldsymbol{P}_{t|t}$.

Indeed, a single prediction-update step for the standard KF has cubic time complexity in the state dimension, p [4]. As such, if p is large, this motivates the idea of perfoming our filtering in a q-dimensional space, where $q \ll p$, and then try to recover our state estimates in the original p-dimensional space. Indeed, the idea of running the KF in reduced-dimension PCA space is a proven method, as described in a recent review by Li et al. [3].

4 Dynamic Projections

Suppose we wish to update our PCA loadings as we progress through the iterations. This is a reasonable feature since the loadings estimates may become stale if we keep them static after initialisation. However, one issue arises from this; if we change our projection, then the usual update equations do not adapt to this. For example, the update $P_{k+1} = P_k + Q_k$ assumes the representation given by the k-th projection, likewise for updating the state estimate.

The first problem arises if we have a state $\boldsymbol{\beta}$, and assume we have an estimate of it in projected space: $\hat{\boldsymbol{\beta}}^U \approx \boldsymbol{\beta}U$, for $U \in \mathbb{R}^{p\times q}$, $q < p$. Then, suppose we wish to find the estimate in a different projected space, say, $\hat{\boldsymbol{\beta}}^V \approx \boldsymbol{\beta}V$, for $V \in \mathbb{R}^{p\times q'}$, $q' < p$. Additionally, suppose U and V arise from PCA so that $U^TU = I_q$ and $V^TV = I_{q'}$. Since we never observe $\boldsymbol{\beta}$ (or indeed, any estimate in the unprojected space), then we need an 'optimal' mapping from $\hat{\boldsymbol{\beta}}^U$ to $\hat{\boldsymbol{\beta}}^V$. We formalise this by the following optimisation problem:

$$\min_{W\in\mathbb{R}^{q\times q'}} \|\boldsymbol{\beta}^U W - \boldsymbol{\beta}^V\|_2$$

This is precisely a least squares problem, and the solution is

$$\begin{aligned} W &= ((\boldsymbol{\beta}^U)^T\boldsymbol{\beta}^U)^{-1}(\boldsymbol{\beta}^U)^T\boldsymbol{\beta}^V \\ &= (U^T\boldsymbol{\beta}^T\boldsymbol{\beta}U)^{-1}U^T\boldsymbol{\beta}^T\boldsymbol{\beta}V \\ &= U^TV \end{aligned}$$

So the least-square optimal state representation in V space is $\hat{\boldsymbol{\beta}}^V = \hat{\boldsymbol{\beta}}^U U^T V$. One can intuitively think of this as a mapping chain from $\mathbb{R}^q \xrightarrow{(1)} \mathbb{R}^p \xrightarrow{(2)} \mathbb{R}^{q'}$, where $(1 = U^T)$ tries to optimally 'undo' PCA and reconstruct the native space state, then $(2 = V)$ maps to $\mathbb{R}^{q'}$ via PCA.

We note that every time we apply an approximate projection, we permanently lose information in the direction of the bases lost in the prior projection. As such, we anticipate that this approach should not be chained for an extended number of iterations if the gross shift in the PCA loadings is expected to be very large.

4.1 Rotating Covariance

We propose a modification to the usual structure by allowing the PCA-loadings of the $\boldsymbol{x}$ data to rotate over time, while keeping the component variances (eigenvalues) constant. We can do this by using a block-rotation matrix R with reasonably small angles, and then we create the new covariance matrix via,

$$\Sigma' = R\Sigma R^T$$

Observe that implicitly we are rotating the PCA loadings since $R\Sigma R^T = RQ\Lambda Q^T R^T = Q'\Lambda(Q')^T$, where $Q' = RQ$. To be more precise, R is constructed as $R = \text{diag}(R(\theta_1), \ldots, R(\theta_k))$ for $k = p/2$ if p is even, else $R = \text{diag}(1, R(\theta_1), \ldots, R(\theta_{\lfloor p/2 \rfloor}))$; see [1]. Hence, to rotate the data incrementally, we can start with some covariance matrix Σ_0, then define:

$$\Sigma_n = R^n\Sigma_0(R^n)^T$$

An important property of this transformation is that because the eigenvalues stay intact, the number of components which explain a given proportion of variance stays constant. In the algorithm, we commute R to produce some $R' = ORO^T$ for a random $O \in \mathbb{R}^{p\times p}$ orthogonal, thus randomising the planes for which we rotate about.

There is a question of how to best optimise this for a large dataset. One can directly construct R^n for each n by replacing $R(\theta_j)$ with $R(n\theta_j)$ for each $j = 1, \ldots, k$, hence eliminating the need to compute the matrix power. From there, we can batch multiply to calculate the entire sequence $(\Sigma_1, \ldots, \Sigma_N)$ simulataneously, provided memory constraints are respected. The construction of the block-diagonal rotation matrix can be computationally expensive if done individually for every n (i.e., every observation).

In context of the projection chaining apporach, perhaps it is viable if the accumulated subspace rotation across the dataset is small.

5 Dynamic PCA-KF

This applies the previous projection chaining apporach to the PCA-KF algorithm. The algorithm is given as follows:

1. **Predict:**
   (a) $\boldsymbol{P}_{t|t-1} = \boldsymbol{P}_{t-1|t-1} + \boldsymbol{Q}_{t-1|t-1}$
   (b) $\boldsymbol{\beta}'_{t|t-1} = \boldsymbol{\beta}'_{t-1|t-1}$
2. **Project (if not first iteration):**
   (a) Acquire the new PCA loadings $\boldsymbol{U}_t$.
   (b) Acquire the pushforward map: $\boldsymbol{V}_t = \boldsymbol{U}_{t-1}^T \boldsymbol{U}_t$
   (c) $\boldsymbol{P}_{t|t-1} \leftarrow \boldsymbol{V}_t^T \boldsymbol{P}_{t|t-1} \boldsymbol{V}_t$
   (d) $\boldsymbol{Q}_{t|t-1} \leftarrow \boldsymbol{V}_t^T \boldsymbol{Q}_{t|t-1} \boldsymbol{V}_t$
   (e) $\boldsymbol{\beta}'_{t|t-1} \leftarrow \boldsymbol{\beta}'_{t|t-1} \boldsymbol{V}_t$
   (f) $\boldsymbol{z}_t \leftarrow \boldsymbol{z}_t \boldsymbol{V}_t$
   (g) $\boldsymbol{C}_t \leftarrow \boldsymbol{C}_t \boldsymbol{V}_t$; keep track of the accumulated projection.
3. **Update:**
   (a) $\delta_t = y_t - \boldsymbol{\beta}'^T_{t|t-1} \boldsymbol{z}_t$
   (b) $S_t = \boldsymbol{z}_t^T \boldsymbol{P}_{t|t-1} \boldsymbol{z}_t + R$
   (c) $\boldsymbol{K}_t = S_t^{-1} \boldsymbol{P}_{t|t-1} \boldsymbol{z}_t$
   (d) $\boldsymbol{\beta}'_{t|t} \leftarrow \boldsymbol{\beta}'_{t|t-1} + \delta_t \boldsymbol{K}_t$
   (e) $\boldsymbol{\beta}_{t|t} \leftarrow \boldsymbol{\beta}'_{t|t} \boldsymbol{C}_t^T$; This is the native $\mathbb{R}^p$ state estimate.
   (f) $\boldsymbol{P}_{t|t} \leftarrow (I - \boldsymbol{K}_t \boldsymbol{z}_t^T) \boldsymbol{P}_{t|t-1} (I - \boldsymbol{K}_t \boldsymbol{z}_t^T)^T + R \boldsymbol{K}_t \boldsymbol{K}_t^T$: Joseph Update

We note that the predict and project steps can be swapped to no effect. The predict and update steps are generic to the KF algorithm when using the Joseph form of the update. We use Joseph form due to enhanced numerical accuracy compared to simpler forms of the update step [5].

6 Results

Here we present some simulation results to test the speed and accuracy of the PCA-KF model. In addition, we provide a result which illustrates how 'stale' a static PCA estimate can get when the data is subjected to the covariance rotation as described in Section 4.1.

6.1 Dynamic Projection Tests

First we present a simulation result with the following hyperparamters:

- $(n, n_{burn}, p) = (2500, 400, 200)$, where n is the number of observations, p is the state dimension, and n_{burn} is the number of observations used to initialise PCA.

- Initial PCA proportion of variance retained (target): 0.80.
- Maximum angle increment (in radians) $\theta = 0.005$, or, $\frac{2\pi}{\theta} \approx 1257$.
- $\text{Var}(u_t) = 10^{-2}$
- ε_t construction scale parameter is 10^{-2}
- x_t construction scale parameter is 10^{-2}

We fix an initial PCA rotation L, then at time t (of the non-burn data), we find the proportion of variance explained by L of the true covarinace Σ_t as: $\frac{\text{tr}(L^T \Sigma_t L)}{\text{tr}(\Sigma_t)}$.

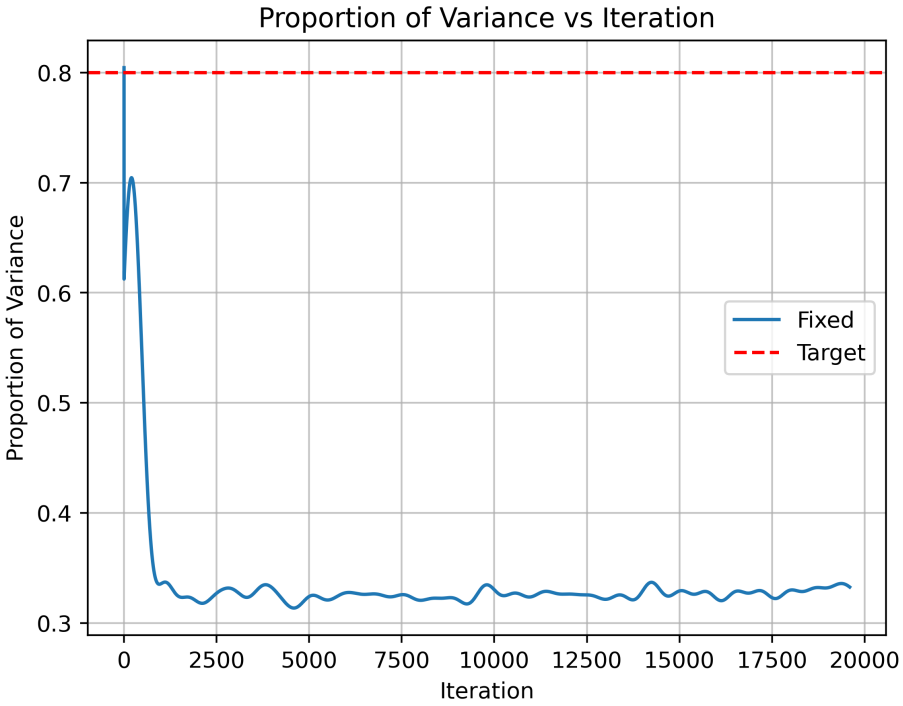


Figure 1: Proportion of variance retained by method.

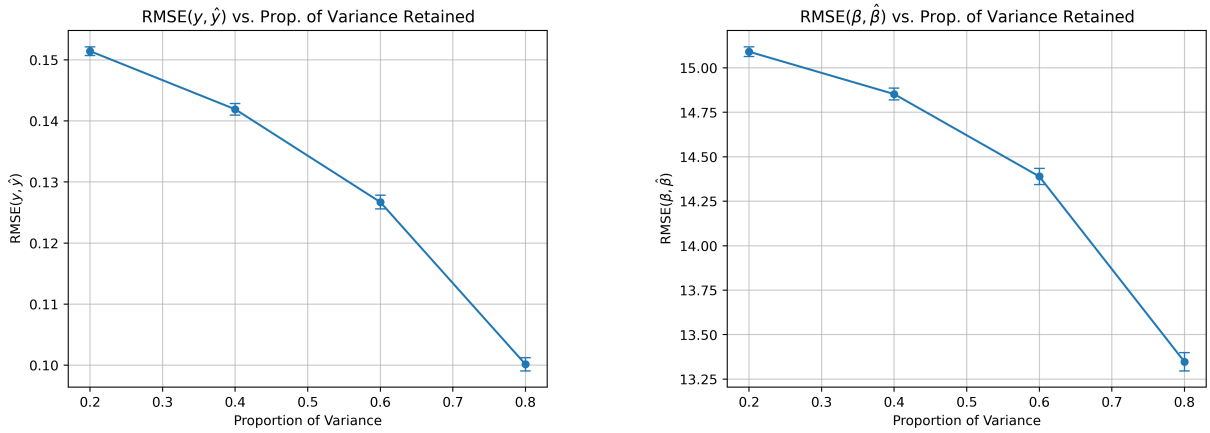
We can observe from Figure 1 that the explained variance of the initial q eigenvectors of the true covariance matrix drops sharply. Indeed, past roughly 1000 iterations, the proportion of variance plateaus at ~ 0.33. This suggests the need to update our representations as we iterate through the algorithm.

6.2 Dynamic Filter

Now, we provide a simulation result for the full PCA-KF algorithm with the following hyperparamters:

- $(n, n_{burn}, p) = (2500, 500, 200)$
- Initial PCA proportion of variance retained (target): 0.90.
- Angle increment (in radians) $\theta = 0.0002$, to prevent excessive shrinkage of our estimate.
- $\text{Var}(z_t) = 10^{-2}$
- ε_t construction scale parameter is 10^{-2}
- x_t construction scale parameter is 10^{-2}

Figure 2: KF estimation error vs proportion of variance

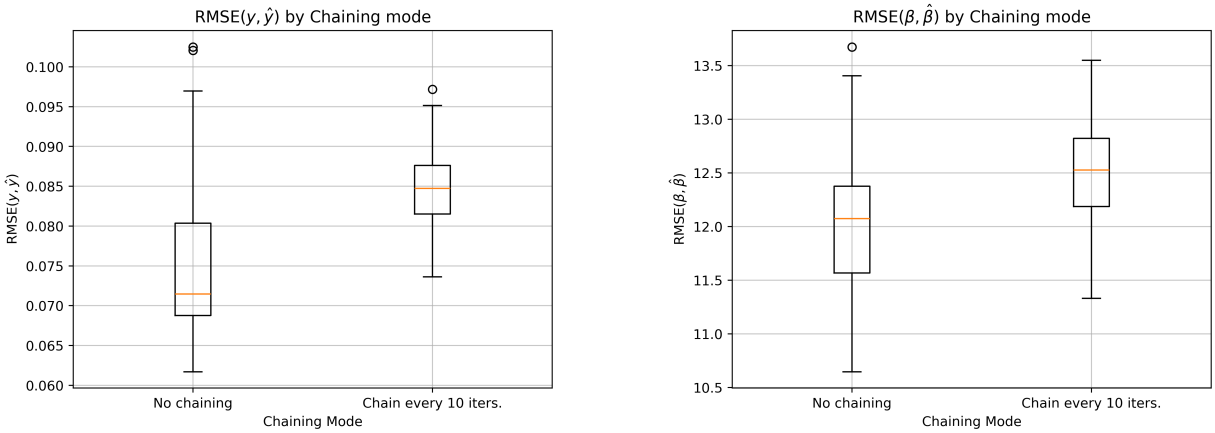


(a) RMSE($\hat{y}, y$) **(b)** RMSE($\hat{\beta}, \beta$)

We see from Figure 2 that increasing the retained proportion of variance reduces the error in both metrics, well beyond the margin of error. However, the actual difference for the $\boldsymbol{\beta}$ RMSE is only increased by $\sim 13\%$ when going from 0.8 to 0.2 proportion of variance.

Finally, we calculate the fit times and number of components retained for $(n, n_{burn}) = (2000, 400)$, prop=0.90, same scale options as previous simulations, for $p \in \{50, 100, 200, 300\}$. The algorithm was run on an Intel i7 12700KF CPU.

Figure 3: KF estimation error vs chaining mode



(a) RMSE($\hat{y}, y$) **(b)** RMSE($\hat{\beta}, \beta$)

From Figure 3, we note that the chaining approach is convincingly less accurate than static regarding the innovation error, but less convincingly so for the state error. Ergo, we see no advantage to chaining projections over just keeping the static PCA.

Table 1: Runtime and Components retained: Average of 5 runs.

p	mean_time $\pm$ std. err. (s)	q
50	0.379 $\pm$ 0.001	17.2
100	0.945 $\pm$ 0.005	32.4
200	3.402 $\pm$ 0.010	59.2
300	6.378 $\pm$ 0.018	80.9

For a clean optimisation comparison, we can compare the fast block diag algorithm with the base one. Averaging 10 calls to each function gives 0.839 ± 0.004s and 1.741 ± 0.007s for the fast and base algorithms respectively, where the error is the standard error.

7 Discussion

One of the key results is that projection chaning is inferior to keeping a static PCA. It appears that the loss of information across the iterations is sufficient that it worsens accuracy, despite it attempting to correct the drift in the projection of the static PCA.

1 AI Declaration

Claude Opus 5.5 was used to write a single if statement to prepend a matrix with a value, and to generate an output filename in 'run_simulation.py'. Where this has been used, it has been labelled with a comment in the code.

2 Reproducing results

The simulations were run with the following hardware:

- **CPU**: Intel Core i7-12700KF (12 cores / 20 threads)
- **RAM**: 2 x 16 GB DDR4-3200
- **OS**: Windows 11 Pro (build 23600)

Note that in the following scripts, the seed is 123 by default (the same one used to generate the results). A seed can be set manually for 'rotation_test.py' and 'run_simulations.py' via the '–seed' argument.

Figure 1 can be reproduced via the following command:

```
python rotation_test.py --n 20000 --p 200 --burn 400 --prop 0.80 --max-angle 0.005 --ts
```

Figures 2 and 3 can be reproduced by first running the following simulation commands:

```
python run_simulation.py --n 2500 --p 200 --burn 500 --n-iters 200 --rotation-angle 0.0002 --
rotate-cov --ts --prop x
```

for $x \in \{0.2, 0.4, 0.6, 0.8\}$, and the following for the chain/no-chain test,

```
python run_simulation.py --n 2500 --p 200 --burn 500 --n-iters 50 --prop 0.90 --rotation-angle
0.0002 --rotate-cov --ts --prop 0.9
```

```
python run_simulation.py --n 2500 --p 200 --burn 500 --n-iters 50 --prop 0.90 --chain --chain-
every 10 --rotation-angle 0.0002 --rotate-cov --ts --prop 0.9
```

Then to generate the four plots, run:

```
python get_sim_plots.py
```

The data for Table 1 can be generated via running the following commands individually:

```
python run_simulation.py --n 2000 --p 50 --burn 400 --prop 0.80 --rotation-angle 0.001 --rotate-
cov --time --outfolder "data/time_benchmarks" --n-iters 5
python run_simulation.py --n 2000 --p 100 --burn 400 --prop 0.80 --rotation-angle 0.001 --rotate-
cov --time --outfolder "data/time_benchmarks" --n-iters 5
python run_simulation.py --n 2000 --p 200 --burn 400 --prop 0.80 --rotation-angle 0.001 --rotate-
cov --time --outfolder "data/time_benchmarks" --n-iters 5
python run_simulation.py --n 2000 --p 300 --burn 400 --prop 0.80 --rotation-angle 0.001 --rotate-
cov --time --outfolder "data/time_benchmarks" --n-iters 5
```

The data for the block diagonal comparison can be generated via the following command, and it prints to the terminal:

```
python run_simulation.py --n 3000 --p 200 --burn 400 --prop 0.80 --rotation-angle 0.001 --rotate-
cov --time --outfolder "data/time_benchmarks" --n-iters 1 --bench
```

Bibliography

[1] rschwieb (https://math.stackexchange.com/users/29335/rschwieb). *Define Rotation matrices for arbitrary dimension?* Mathematics Stack Exchange. URL:https://math.stackexchange.com/q/386513 (version: 2013-05-10). eprint: https://math.stackexchange.com/q/386513. URL: https://math.stackexchange.com/q/386513.

[2] Rudolf E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.

[3] Guo Li et al. "High-dimensional Kalman filtering: From sparse modeling to AI-augmented architectures". en. In: *Signal Processing* 250 (Jan. 2026), p. 110785. ISSN: 01651684. DOI: 10.1016/j.sigpro.2026.110785. URL: https://linkinghub.elsevier.com/retrieve/pii/S0165168426002987.

[4] Jonathan Schmidt et al. *The Rank-Reduced Kalman Filter: Approximate Dynamical-Low-Rank Filtering In High Dimensions*. 2024. arXiv: 2306.07774 [stat.ML]. URL: https://arxiv.org/abs/2306.07774.

[5] Renato Zanetti and Kyle J. DeMars. "Joseph Formulation of Unscented and Quadrature Filters with Application to Consider States". en. In: *Journal of Guidance, Control, and Dynamics* 36.6 (Nov. 2013), pp. 1860–1864. ISSN: 0731-5090, 1533-3884. DOI: 10.2514/1.59935. URL: https://arc.aiaa.org/doi/10.2514/1.59935.